MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

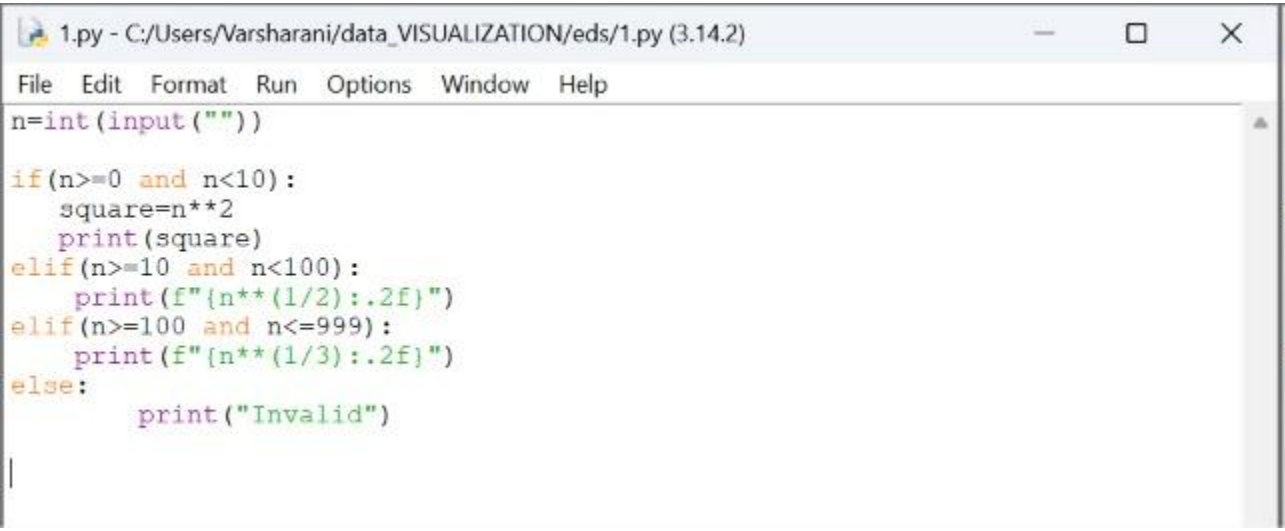
TOPIC : EDS LAB PRACTICAL ASSIGNMENT

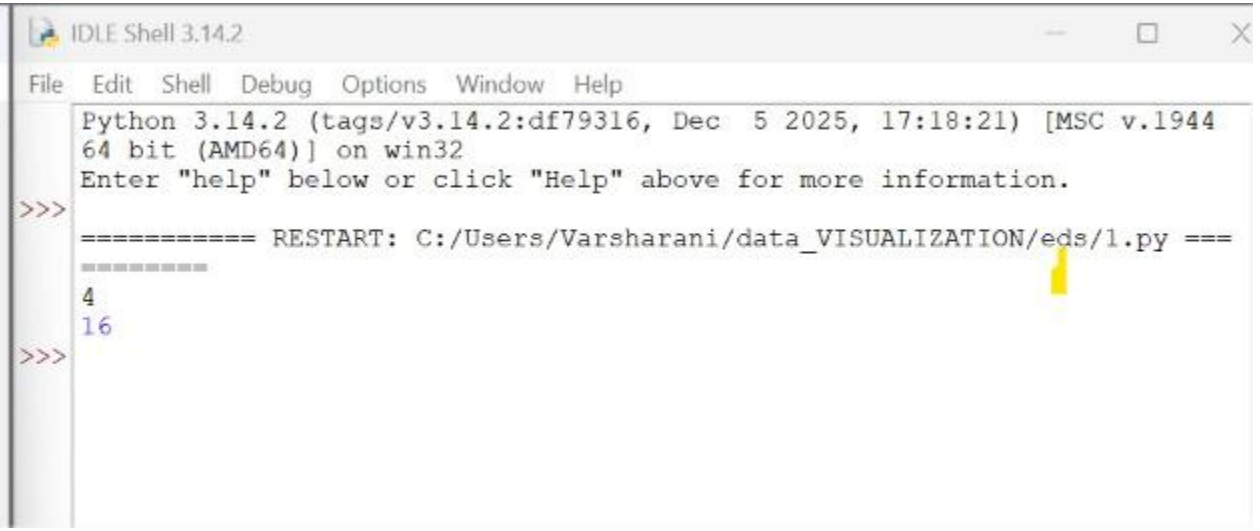
NAME	VARSHARANI SHIVAJI PATIL
PRN	202501100152
SUBJECT	ESSENTIAL OF DATA SCIENCE
DIVISION	SE2
BATCH	S22

PRACTICAL NO. 01

Practice Lab Assignments:

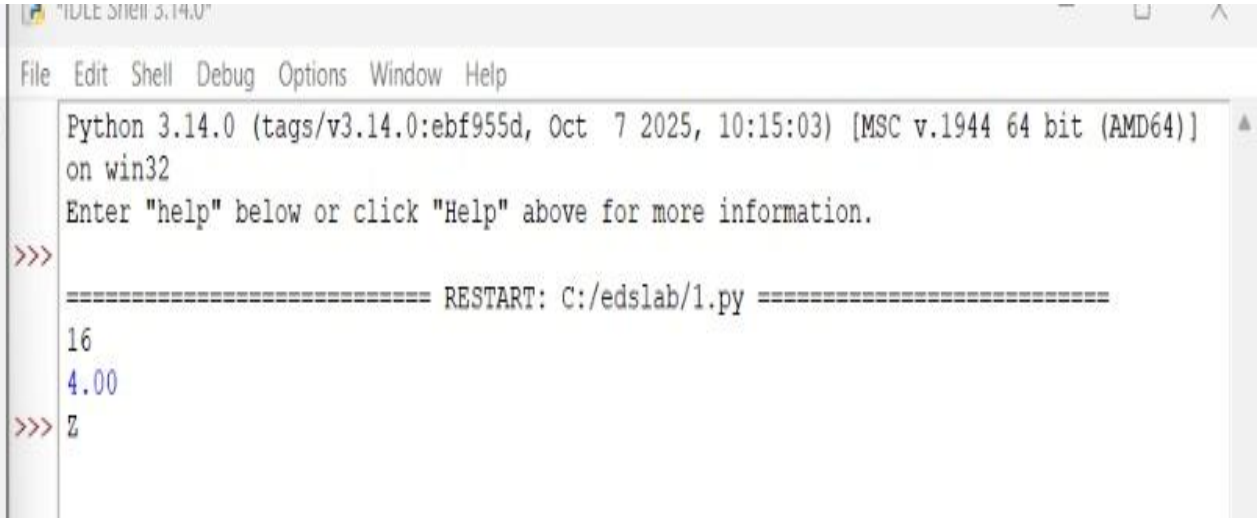
1. To accept an object mass in kilograms and velocity in meters per second and display its momentum. Momentum is calculated as e=mc2 where m is the mass of the object and c is its velocity.



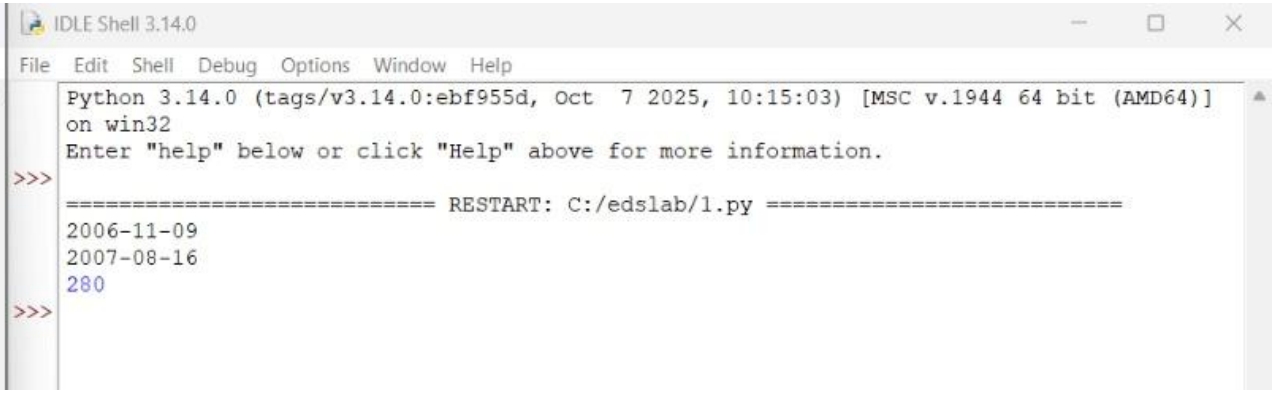


2.Write a Pythonprogram forfollowing conditions.

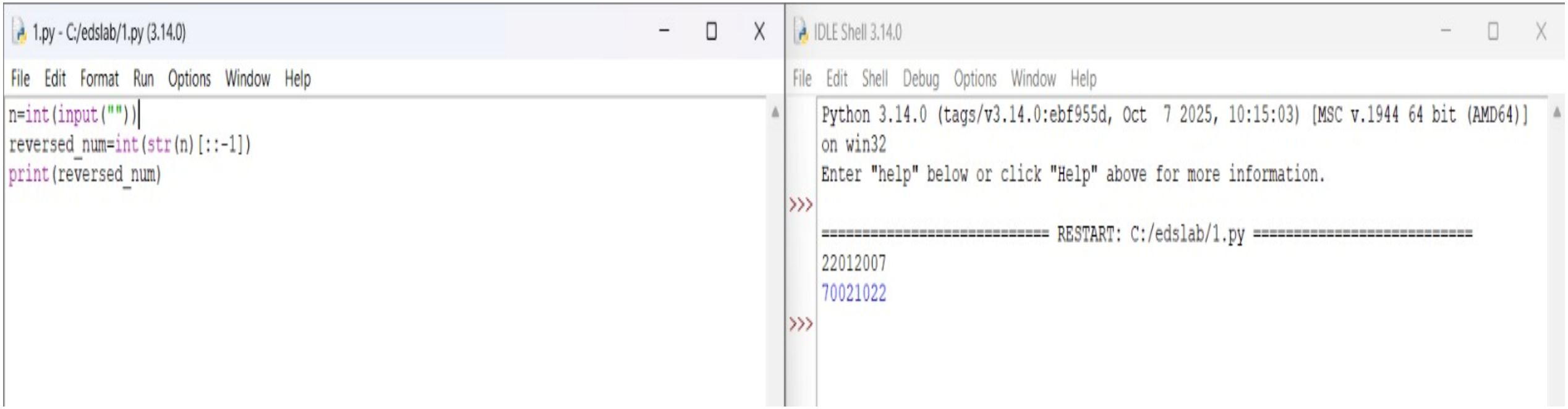
- If n is single digit print square of it.
- If n is two digit print squareroot of it.
- If n is three digit print cube root of it.



3. Read the birth date and salary in rupees of employees. Perform data transformation for birth date to ageand also salary which is in rupees to salary in dollars using functions



4.Print the reverse number of a given number,



Lab Assignment :

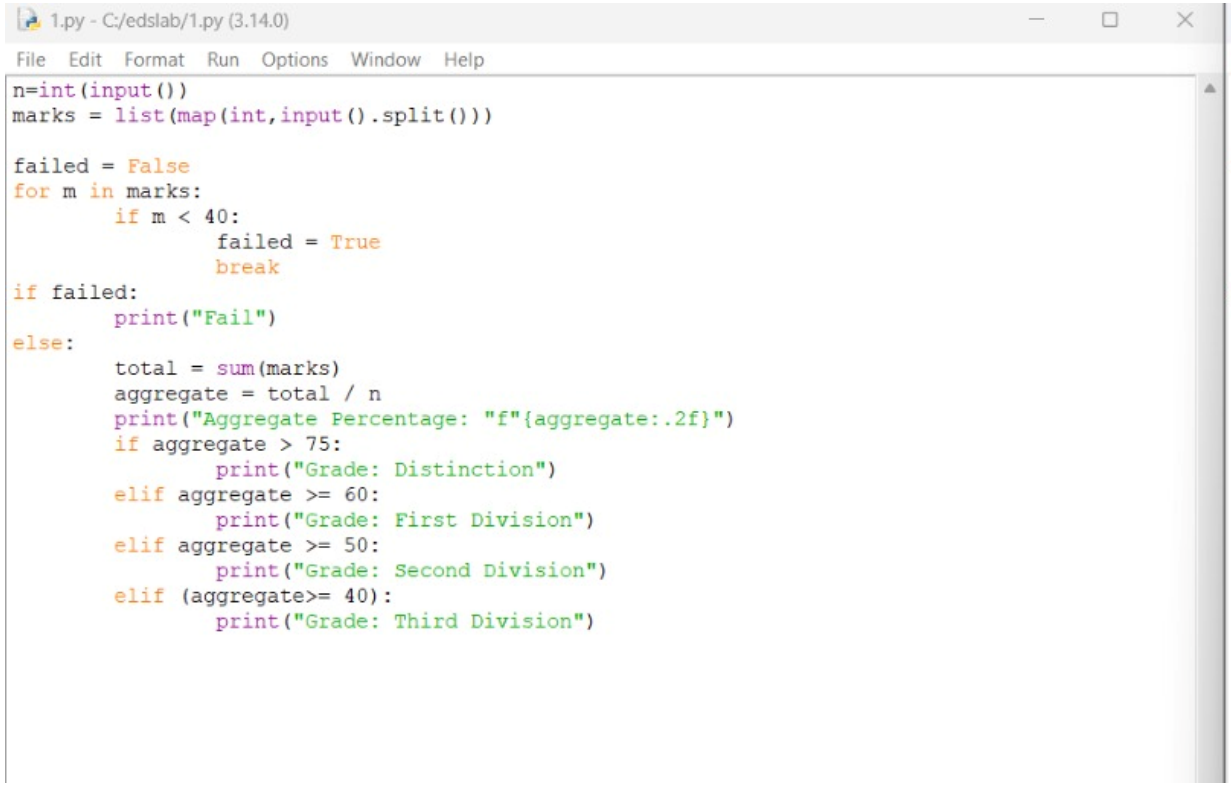
1. To accept students five courses marks and compute his/her result. Student is passing if he/she scores marks equal to and above 40 in each course.

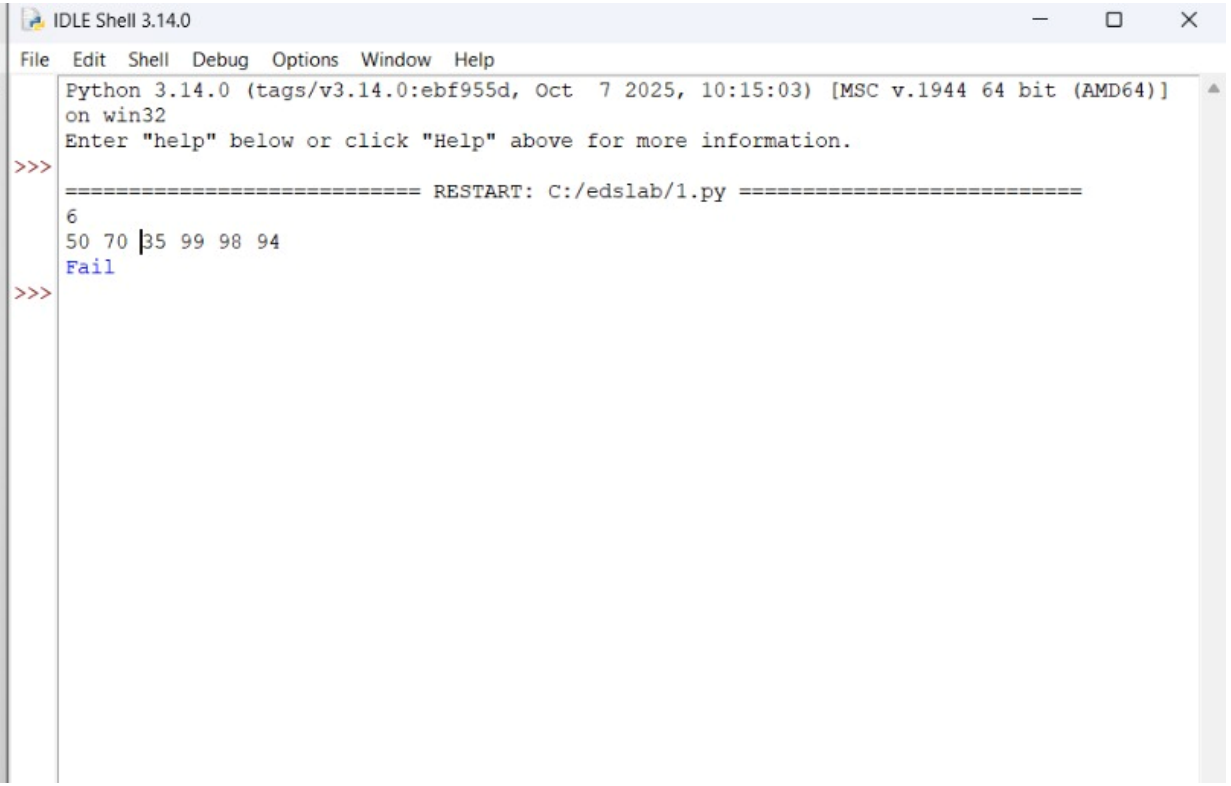
If student scores aggregate greater than 75 percentage , then the grade is distinction.

If aggregate is greater than or equal to 60 and less than 75 then the grade if first division.

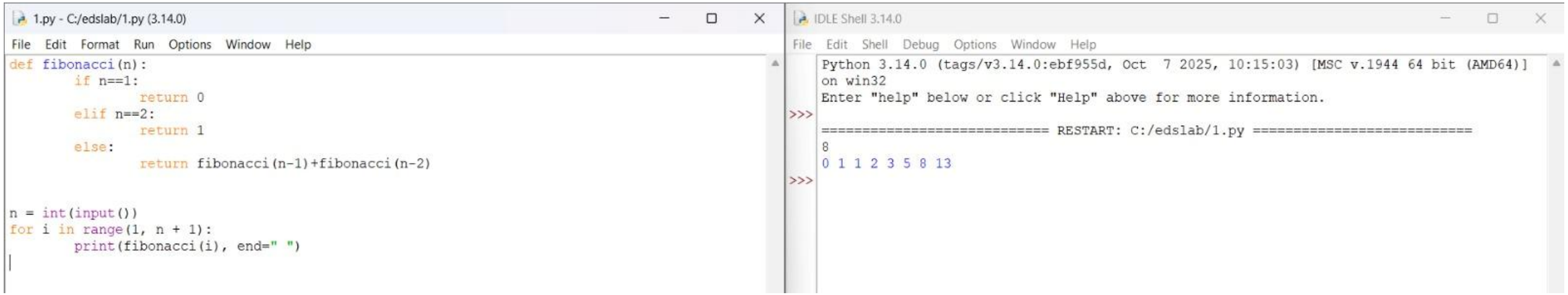
If aggregate is greater than or equal 50 and less than 60 , then the grade is second division.

If aggregate is greater than or equal 40 and less than 50 , then the grade is third division.

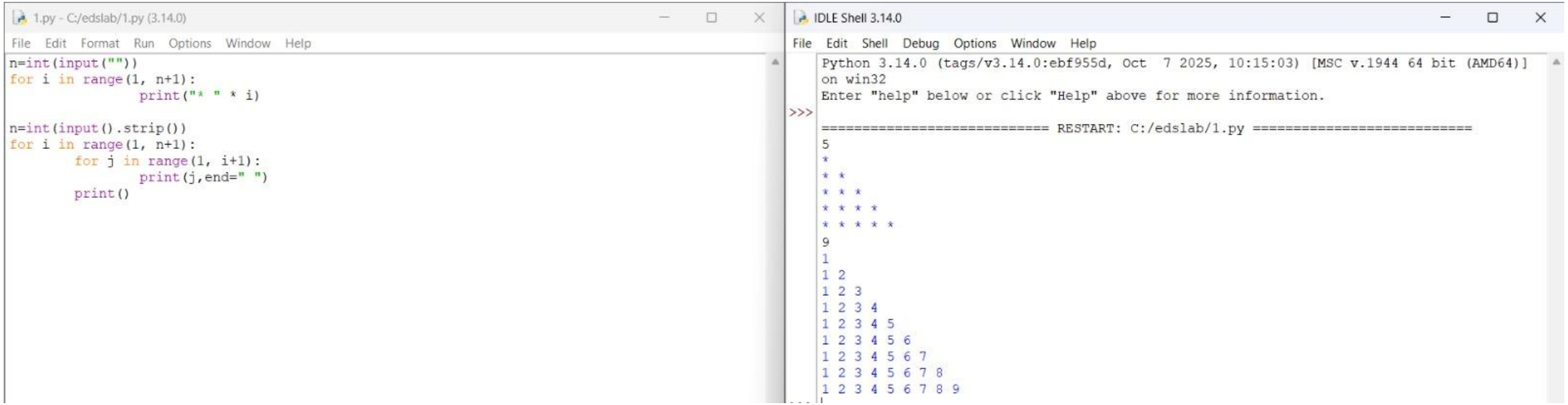




2. Solve the Fibonacci sequence using recursive function in Python.



3 . Write a Python program to print different patterns



PRACTICALNO.02

Practice Lab Assignment :

1.Perform all List Operations, Dictionary Operations.

1.py - C:/edslab/1.py (3.14.0)

File Edit Format Run Options Window Help

```
list=[]
while True:
        print("1. Add")
        print("2. Remove")
        print("3. Display")
        print("4. Quit")
        n = int(input("Enter choice: "))
        if n == 1:
                add = int(input("Integer: "))
                list.append(add)
                print(f"List after adding: {list}")
        elif n == 2:
                if len(list) == 0:
                        print("List is empty")
                elif len(list) != 0:
                        remove=int(input("Integer: "))
                        if remove not in list :
                                print("Element not found")
                        else:
                                list.remove(remove)
                                print(f"List after removing: {list}")
        elif n == 3:
                if len(list) == 0:
                        print("List is empty")
                else:
                        print(list)
        elif n == 4:
                break
        else:
                print("Invalid choice")
Initial dictionary
```

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 299
List after adding: [299]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 299
Invalid choice
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
[299]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 2
Integer: 299
List after removing: []
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 4
```

2.Perform all Dictionary Operations

```
# Initial dictionary
student = {
    1: "Amit",
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}
print("Original Dictionary:",student)
key1=int(input())
value1=input()
student.update({key1:value1})
print("After Insertion:",student)
key2=int(input())
value2=input()
if key2 in student.keys():
        student.update({key2:value2})
print("After Update:",student)
key3=int(input())
if key3 in student.keys():
        student.pop(key3)
print("After Deletion:",student)
print("Traversing Dictionary:")
for key, value in student.items():
        print(f"{key} : {value}")
```

```
2. Remove
3. Display
4. Quit
Enter choice: 4
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja
', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
varsharani
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7
: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'varsharani'}
8
pranjali
After Update: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: '
Rahul', 8: 'pranjali', 9: 'Vikram', 10: 'Anjali', 11: 'varsharani'}
2
After Deletion: {1: 'Amit', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8
: 'pranjali', 9: 'Vikram', 10: 'Anjali', 11: 'varsharani'}
Traversing Dictionary:
1 : Amit
3 : Kiran
4 : Neha
5 : Arjun
6 : Pooja
7 : Rahul
8 : pranjali
9 : Vikram
10 : Anjali
11 : varsharani
>>> 1
1
```

Lab Assignment :

1. Select the number from the entered list and find its position in Python (use Linear Search)

1.py - C:/edslab/1.py (3.14.0)

File Edit Format Run Options Window Help

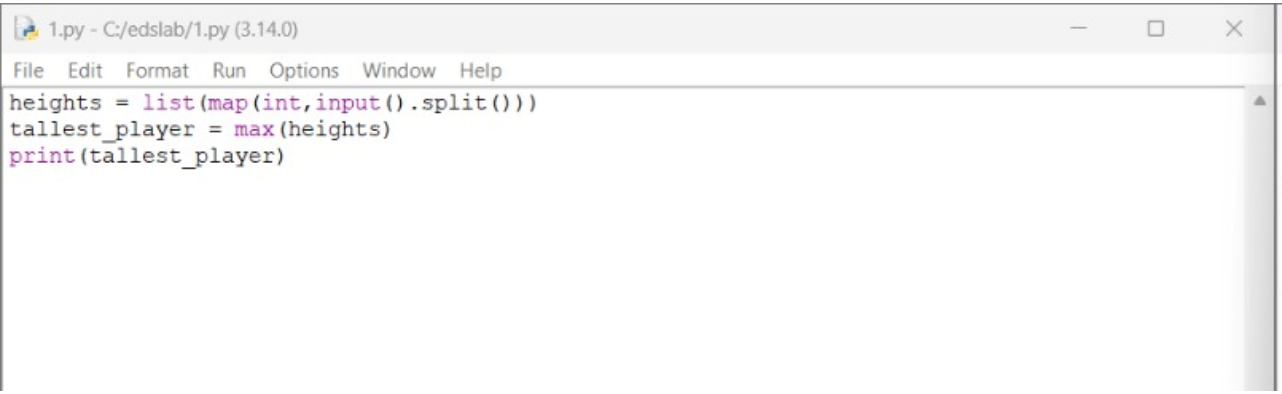
```
def linear_search(arr,key):
        for i in range(len(arr)):
                if arr[i] == key:
                        return i
        return -1
arr = list(map(int,input().split()))
key = int(input())
result = linear_search(arr,key)
if result != -1:
        print(result)
else:
        print("Not found")
```

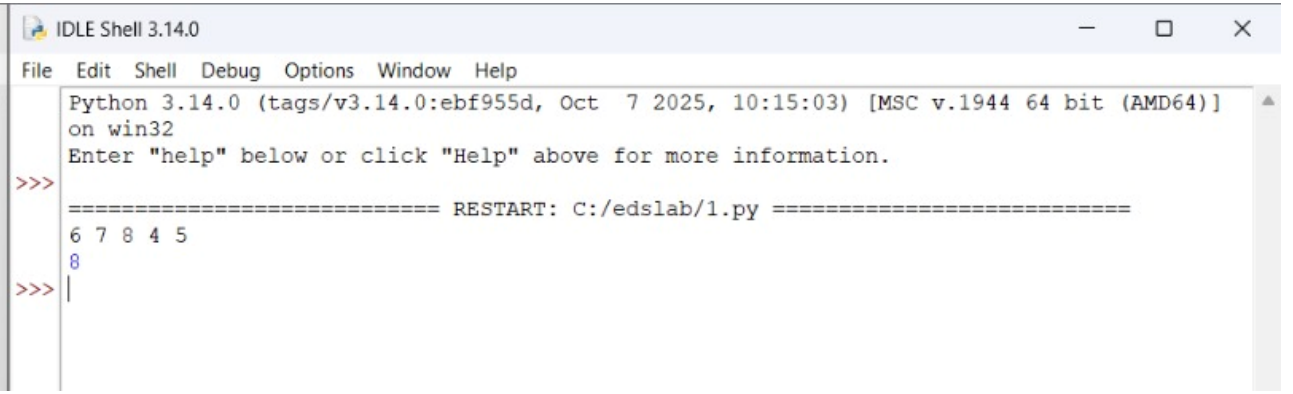
IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/edslab/1.py ==========
1 2 3 4 5 6 7 8 9 0
9
8
>>>
```

2. Choose cricket team of eleven players find the captain of the team (consider tallest person as a captain) using dictionary.

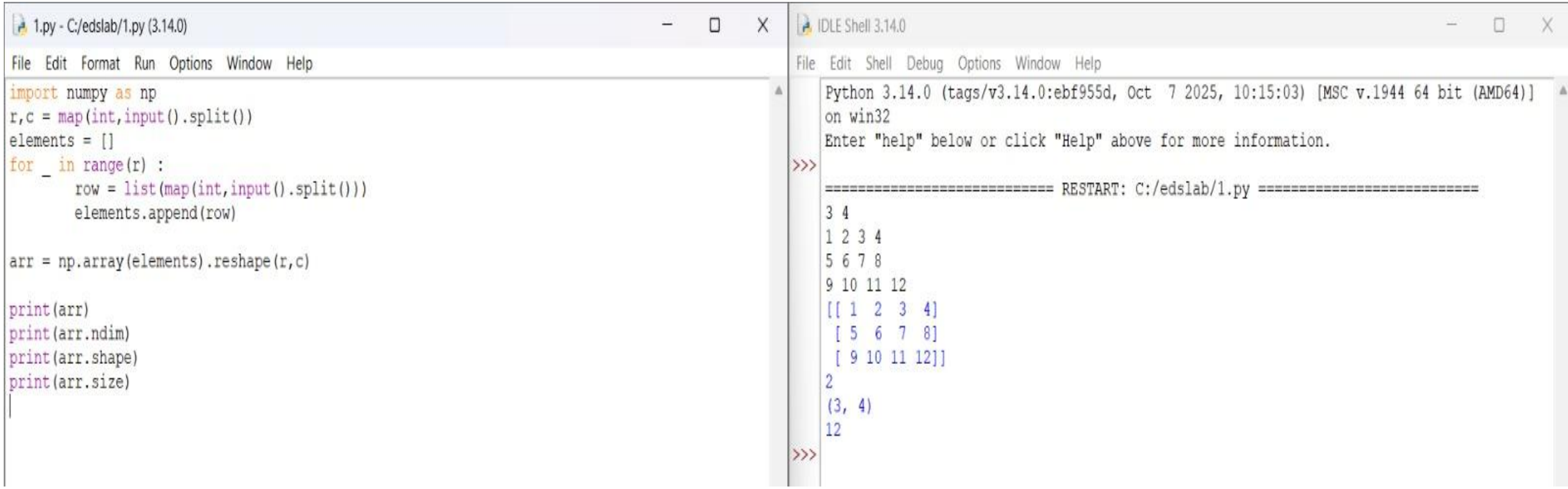




PRACTICAL NO. 03

Practice Lab Assignment:

1. Perform all the NumPy operations in Python.



Lab Assignment:

Prepare/Take dataset for any real life application. Read a dataset into an array. Perform following operations on it as:

- Perform all matrix operations
- Horizontal and vertical stacking of NumPy Arrays

1.py - C:/edslab/1.py (3.14.0)

File Edit Format Run Options Window Help

```
import numpy as np

# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])


# Addition
print("Addition (A + B):")
print(matrix_a + matrix_b)
# Subtraction
print("Subtraction (A - B):")
print(matrix_a - matrix_b)
# Multiplication (element-wise)
print("Element-wise Multiplication (A * B):")
print(matrix_a*matrix_b)
# Matrix multiplication (dot product)
print("A dot B:")
print(np.dot(matrix_a,matrix_b))
# Transpose
print("Transpose of A:")
print(matrix_a.T)
```

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/edslab/1.py ==========
Enter Matrix A:
1 2 3
4 5 6
7 8 9
Enter Matrix B:
1 1 1
2 2 2
3 3 3
Addition (A + B):
[[ 2  3  4]
 [ 6  7  8]
 [10 11 12]]
Subtraction (A - B):
[[0 1 2]
 [2 3 4]
 [4 5 6]]
Element-wise Multiplication (A * B):
[[ 1  2  3]
 [ 8 10 12]
 [21 24 27]]
A dot B:
[[14 14 14]
 [32 32 32]
 [50 50 50]]
Transpose of A:
[[1 4 7]
 [2 5 8]
 [3 6 9]]
>>>
```

- Custom sequence generation

1.py - C:/edslab/1.py (3.14.0)

File Edit Format Run Options Window Help

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
a=np.hstack((arr1,arr2))
print("Horizontal Stack:")
print(a)



# Perform vertical stacking (vstack)
b=np.vstack((arr1,arr2))
print("Vertical Stack:")
print(b)
```

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/edslab/1.py ==========
Enter Array1:
1 2 3
4 5 6
7 8 9
Enter Array2:
4 5 6
7 8 9
10 11 12
Horizontal Stack:
[[ 1  2  3  4  5  6]
 [ 4  5  6  7  8  9]
 [ 7  8  9 10 11 12]]
Vertical Stack:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
>>>
```

- Arithmetic and Statistical Operations

1.py - C:/edslab/1.py (3.14.0)

File Edit Format Run Options Window Help

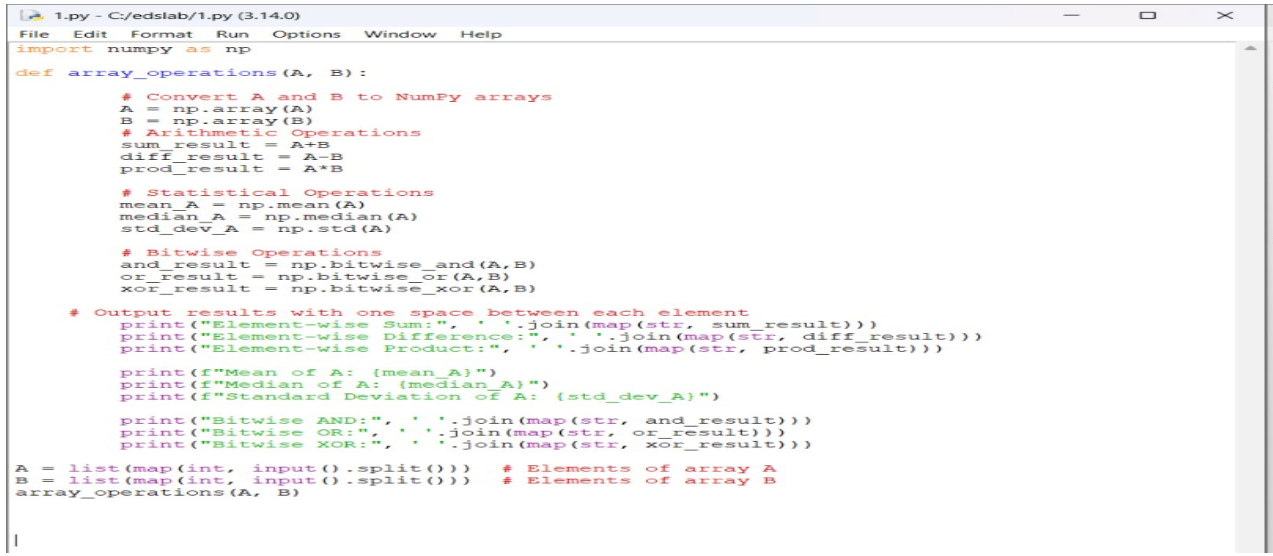
```
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())

# Generate the sequence using np.arange()
sequence = np.arange(start,stop,step)

# Print the generated sequence
print(sequence)
```

IDLE Shell 3.14.0

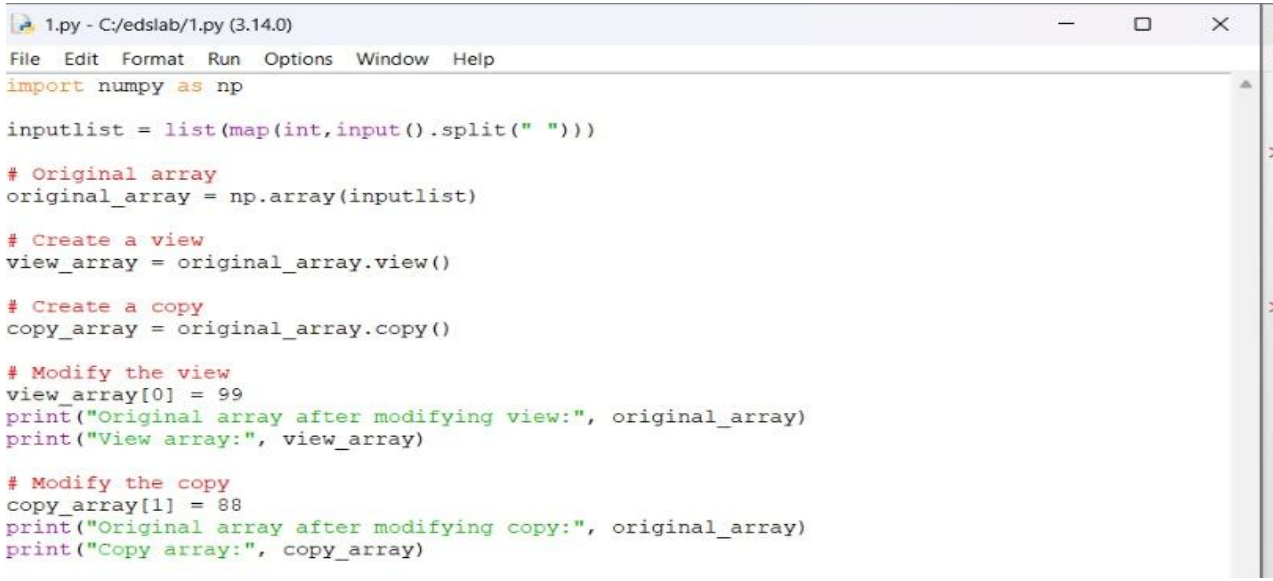
File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/edslab/1.py ==========
3
10
2
[3 5 7 9]
>>>
```

- Bitwise Operators



- Copying and viewing arrays



- Data Stacking
- Searching
- Sorting
- Broadcasting
- Counting

1.py - C:/edslab/1.py (3.14.0)

File Edit Format Run Options Window Help

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split())))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1
a=np.where(array1==search_value)[0]
print(a)
# Count occurrences in array1
b=np.count_nonzero(array1==count_value)
print(b)
# Broadcasting addition
c=array1 + broadcast_value
print(c)
# Sort the first array
d=np.sort(array1)
print(d)
```

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

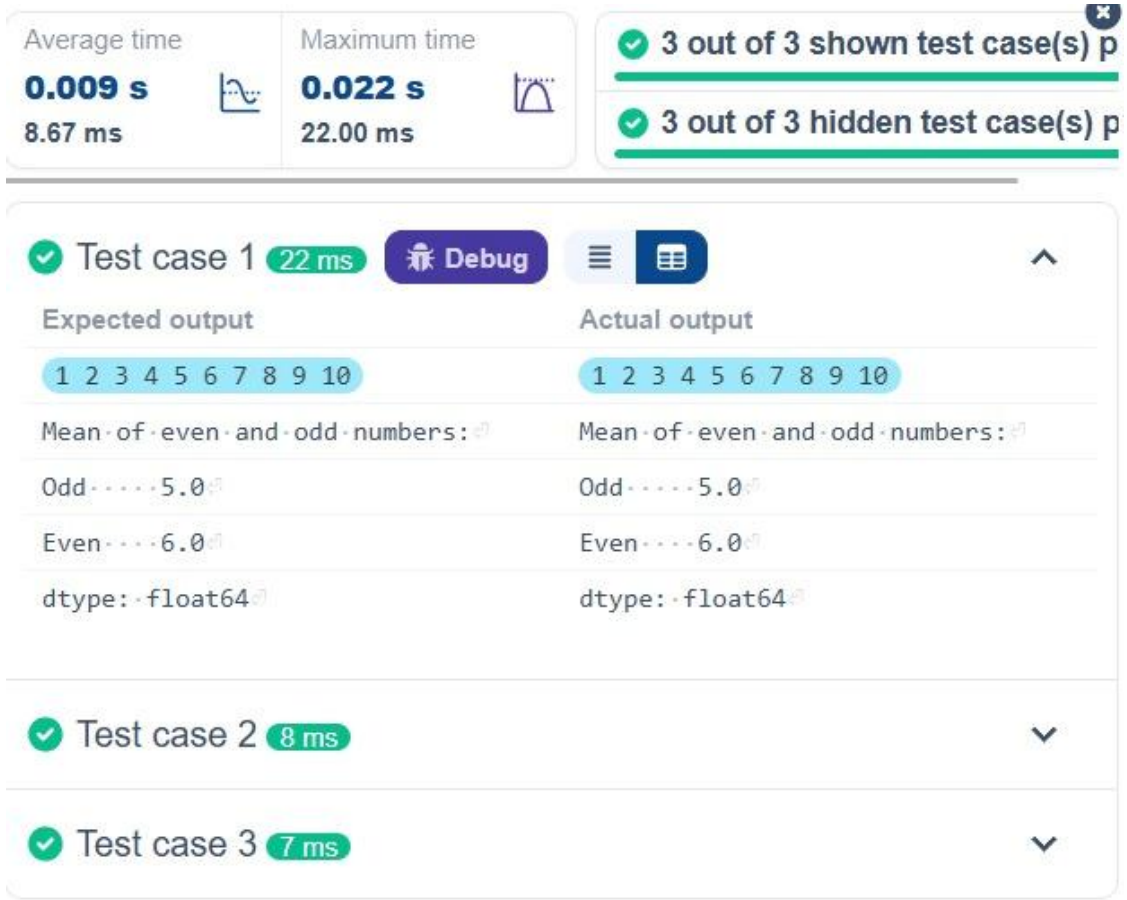
```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/edslab/1.py ==========
1 1 1 2 2 2
Value to search: 1
Value to count: 2
Value to add: 2
[0 1 2]
3
[3 3 3 4 4 4]
[1 1 1 2 2 2]
>>>
```

PRACTICAL NO. 04

Practice Lab Assignment:

1. Perform all the pandas operations in Python.

```python
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)
# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```



Lab Assignment:

Read any real life dataset. Store the data into Data Frames. Identify 10 grains for the given dataset. Implement all 20 grains using Pandas methods.

```
import pandas as pd

# Provided dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
}

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new row
Name=input("New name: ")
Age=int(input("New age: "))
new={'Name':Name,'Age':Age}
df=df.append(new,ignore_index=True)

# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
index=int(input("Index of row to modify: "))
age=int(input("New age: "))
df.at[index,"Age"]=age
```

```
# Deleting a row
delt=int(input("Index of row to delete: "))
df=df.drop(delt).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
genders = input("Enter genders separated by space: ").split()
df["Gender"] = genders

# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name']=df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)

# Deleting a column
df=df.drop(columns=['Age'])
# Display the DataFrame after deleting a column
print("After deleting a column:")
print(df)
```

Average time	Maximum time
0.067 s	**0.071 s**
67.00 ms	71.00 ms

✅ 1 out of 1 shown test case(s) passed

✅ 1 out of 1 hidden test case(s) passed

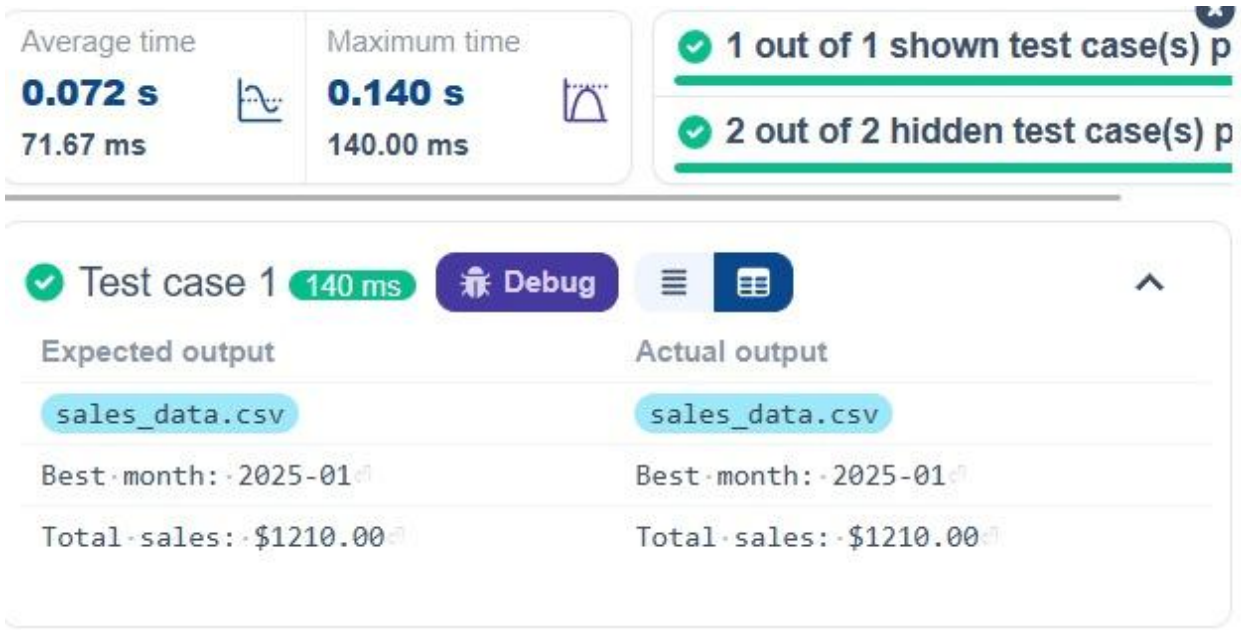
```
New age: 27
After modifying a row:
      Name  Age
0     Alice   27
1       Bob   30
2   Charlie   35
3     Susan   29
Index of row to delete: 3
After deleting a row:
      Name  Age
0     Alice   27
1       Bob   30
2   Charlie   35
Enter genders separated by space: F M M
After adding a new column:
      Name  Age Gender
0     Alice   27      F
1       Bob   30      M
2   Charlie   35      M
After modifying a column:
      Name  Age Gender
0     ALICE   27      F
1       BOB   30      M
2   CHARLIE   35      M
After deleting a column:
      Name Gender
```


```
New age: 27
After modifying a row:
      Name  Age
0     Alice   27
1       Bob   30
2   Charlie   35
3     Susan   29
Index of row to delete: 3
After deleting a row:
      Name  Age
0     Alice   27
1       Bob   30
2   Charlie   35
Enter genders separated by space: F M M
After adding a new column:
      Name  Age Gender
0     Alice   27      F
1       Bob   30      M
2   Charlie   35      M
After modifying a column:
      Name  Age Gender
0     ALICE   27      F
1       BOB   30      M
2   CHARLIE   35      M
After deleting a column:
      Name Gender
```

The Sample Grains for Sales Dataset as:

Which was the best month for sales? How much was earned that month?

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)
df['Date']=pd.to_datetime(df['Date'])
df['Month']=df['Date'].dt.to_period('M').astype(str
)

# Find the month with the highest total sales
df['Sales']=df['Quantity']*df["Price"]
monthly_sales=df.groupby('Month')['Sales'].sum()
best_month =monthly_sales.idxmax()
highest_sales = monthly_sales.max()

print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```



Which product sold the most? Why do you think it did?

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)
product_sales=df.groupby('Product')
['Quantity'].sum().reset_index()

# Find the product with the highest total quantity
sold
best_product
=product_sales.loc[product_sales['Quantity'].idxmax
(),'Product']
highest_quantity = product_sales['Quantity'].max()

# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

```
sales_data.csv

Best selling product: Product A

Total quantity sold: 23
```

Which city sold the most products?

```python
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

city_wise_sales=df.groupby('City')['Quantity'].sum()

# write the code..
best_city= city_wise_sales.idxmax()
# Display the result
print(f"City sold the most products: {best_city}")
```

sales_data.csv

```
City sold the most products: Los Angeles
```

What Products are most often sold together?

```python
import pandas as pd
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)
grouped = df.groupby("Date")["Product"].apply(list)
pairs_counter = Counter()
for products in grouped:
    pairs = combinations(sorted(products), 2)
    pairs_counter.update(pairs)
max_frequency = max(pairs_counter.values())
most_common_pairs = [(pair, freq) for pair , freq in
pairs_counter.items() if freq == max_frequency]
for (product1,product2), frequency in most_common_pairs:
    print(f"{product1} and {product2}: {frequency} times")
```

sales_data.csv

```
Product A and Product B: 2 times
Product A and Product C: 2 times
```

PRACTICAL NO. 05

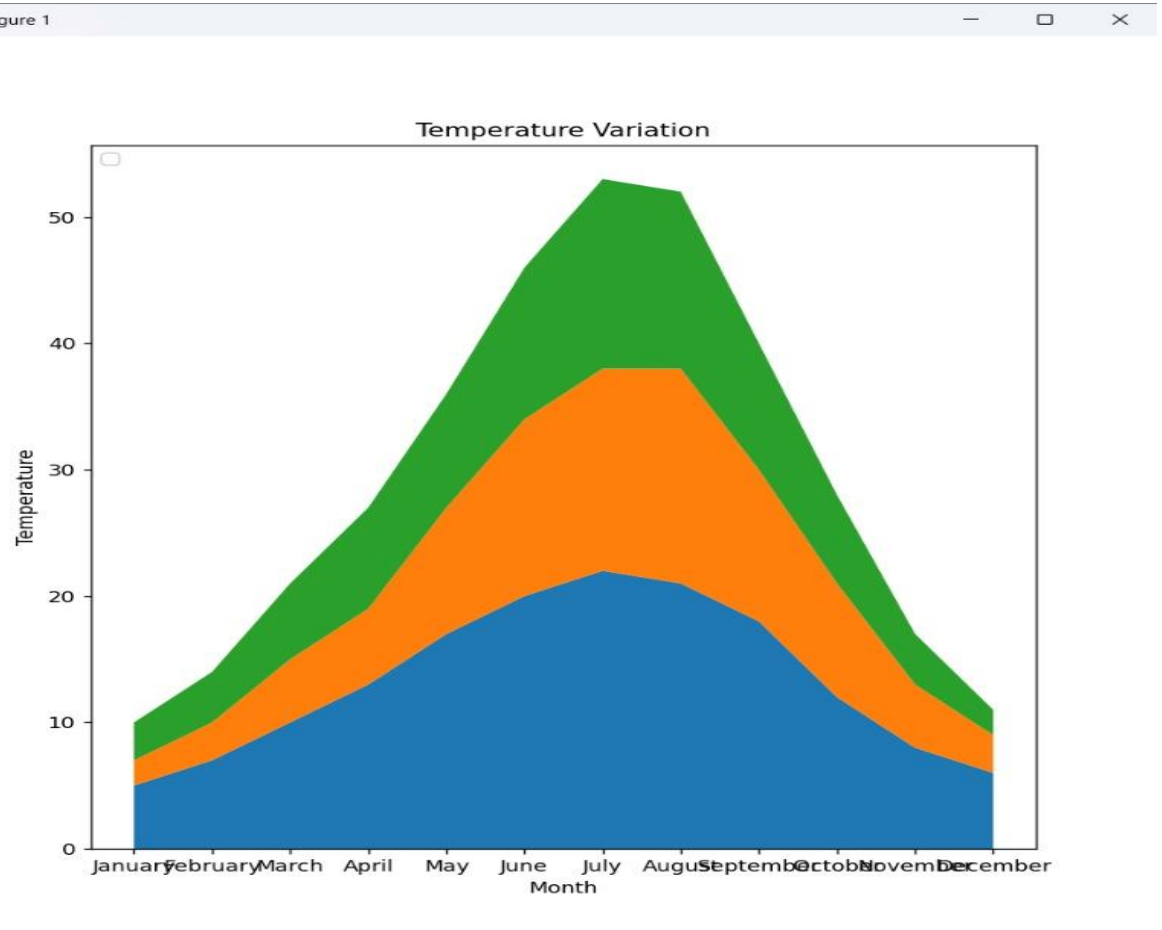
Practice Lab Assignment:

Install MatPlotLib library. Draw basic graphs for sales dataset using MatPlotLib.

1.py - C:/edslab/1.py (3.14.0)

File Edit Format Run Options Window Help

```
import matplotlib.pyplot as plt
import pandas as pd

# Data for Months and Temperature for three cities
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'Sept
    'City_A_Temperature': [5, 7, 10, 13, 17, 20, 22, 21, 18, 12, 8, 6],
    'City_B_Temperature': [2, 3, 5, 6, 10, 14, 16, 17, 12, 9, 5, 3],
    'City_C_Temperature': [3, 4, 6, 8, 9, 12, 15, 14, 10, 7, 4, 2]
}

# Write your code...
df = pd.DataFrame(data)

plt.stackplot(df['Month'],df['City_A_Temperature'],df['City_B_Temperature'],df['City_C_Tempe

plt.xlabel('Month')
plt.ylabel('Temperature')
plt.title('Temperature Variation')
plt.legend(loc='upper left')
plt.show()#
```



Lab Assignment:

For Titanic dataset, Perform data analysis. Identify 5 grains for a given dataset.

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset from the CSV file
df = pd.read_csv('titanic.csv')

# Set up the figure for 5 subplots
fig, axes = plt.subplots(3, 2, figsize=(12, 12))

# write the code.
# Plot 1: Count of passengers by class
axes[0,0].bar(df['Pclass'].value_counts().index,df['Pclass'].value_counts
( ), color='skyblue')
axes [0, 0].set_title("Passenger Class Distribution")
axes [0, 0].set_xlabel("Pclass")
axes [0, 0].set_ylabel("Count")

# Plot 2: Gender distribution
axes[0,1].pie(df['Gender'].value_counts(),labels=df['Gender'].value_count
s().index, autopct='%1.1f%%', colors= ['lightblue', 'lightcoral'])
axes[0, 1].set_title("Gender Distribution")

# Plot 3: Age distribution
axes[1, 0].hist(df['Age'].dropna(), bins=8, color='lightgreen',
edgecolor='black')
axes[1, 0].set_title("Age Distribution")
axes[1, 0].set_xlabel("Age")
axes[1, 0].set_ylabel("Frequency")
```

```
# Plot 2: Gender distribution
axes[0,1].pie(df['Gender'].value_counts(),labels=df['Gender'].value_count
s().index, autopct='%1.1f%%', colors= ['lightblue', 'lightcoral'])
axes[0, 1].set_title("Gender Distribution")

# Plot 3: Age distribution
axes[1, 0].hist(df['Age'].dropna(), bins=8, color='lightgreen',
edgecolor='black')
axes[1, 0].set_title("Age Distribution")
axes[1, 0].set_xlabel("Age")
axes[1, 0].set_ylabel("Frequency")

# Plot 4: Survival count
axes[1, 1].bar(df['Survived'].value_counts().index,
df['Survived'].value_counts(), color=['lightblue', 'lightcoral'])
axes[1, 1].set_title("Survival Count")
axes[1, 1].set_xlabel("Survived (0 = No, 1 = Yes)")
axes[1, 1].set_ylabel("Count")

# Plot 5: Fare vs Age
axes[2, 0].scatter(df['Age'], df['Fare'], color='orange',
edgecolors='black')
axes[2, 0].set_title("Fare vs Age")
axes[2, 0].set_xlabel("Age")
axes[2, 0].set_ylabel("Fare")

plt.tight_layout()
plt.show()
```

Perform a data visualization for those grains using the MatPlotLib library. (Use any 5 different graphs with proper title, legends, axis names, etc. to map identified grains)